



Advanced Programming Interfaces

What is an interface?

Webster: “a place at which *independent and often unrelated systems* meet and communicate with each other“, respecting a set rules .



Contract, Protocol

- An interface describes a model, a contract.
- A class may implement that model, adhering to the contract and strictly respecting its specifications.



Defining an Interface

```
[public] interface InterfaceName
    [extends SuperInterface1, SuperInterface2...] {
        /* Constant declarations
           Abstract methods
           Default methods
           Static methods */
    }
```

Example:

```
public interface Student {
    double MAX_GRADE = 5;
    void sayHello();
    double getExamGrade(Exam exam);
}
```

 //public static final ...
//public abstract ...

Implementing an Interface

```
class ClassName implements Interface1, Interface2, ... {  
    /* A concrete class that implements an interface  
       must specify code for all abstract methods  
       declared by the interface */  
}
```

Example:

```
public class ComputerScienceStudent implements Student {  
    private String name;  
    public boolean isProgrammer() { return true; }  
    ...  
    public void sayHello() {  
        System.out.println("Hello, my name is " + name);  
    }  
    public double getExamGrade(Exam exam) {  
        return MAX_GRADE;  
    }  
}
```

Common Denominator

- Suppose we already have class hierarchies
 - Person *extends* Entity
 - Parrot *extends* Animal
 - Alexa *extends* Robot
- All of them can “talk”
 - Person *extends* Entity **implements** Talkative
 - Parrot *extends* Animal **implements** Talkative
 - Alexa *extends* Robot **implements** Talkative
- **Talkative[]** noisy = {johnny, coco, alexa}

```
public interface Talkative {  
    void talk();  
}
```

Interface – Reference Type

We may say that an object is of type X, where X is an interface, if that object is an instance of a class implementing the interface X.

```
Student student = new ComputerScienceStudent();  
    student.sayHello(); student.isProgrammer();  
Student student = new Student();
```

```
Talkative johnny = new Person();    johnny.talk();  
Talkative coco = new Parrot();      coco.talk();  
Talkative = new Talkative();
```

```
AutoCloseable reader = new FileReader("fis.txt"); reader.close();  
AutoCloseable con = DriverManager.getConnection(); con.close();  
AutoCloseable reader = new AutoCloseable();
```

Multiple Implementations

```
public interface Matrix {
    void set(int row, int col, double value);
    double get(int row, int col);
    Matrix add(Matrix m);
    Matrix mul(Matrix m);
    ...
}

public class DefaultMatrixImpl implements Matrix {
    private double[][] data;
    public DefaultMatrixImpl(int rows, int cols) { ... }
    ...
}

public class SparseMatrixImpl implements Matrix {
    private int[] row;
    private int[] col;
    private double[] data;
    public SparseMatrixImpl(int rows, int cols) { ... }
    ...
}

public static void main ( String args []){
    Matrix a = new DefaultMatrixImpl(10, 10); a.set(0,0, 123); ...
    Matrix b = new SparseMatrixImpl (10, 10); b.set(9,9, 456); ...
    Matrix c = a.add(b);
}
```


Interfaces and Abstract Classes

- Extending an abstract class imposes a strong relationship among two classes.
- Implementing an interface is much lighter: it only specifies that a class is respecting a certain contract, making sure it conforms to some specifications.
- Interfaces and abstract classes do not exclude each other, they are used together in many situations:
 - ♦ **List** → **the contract**
 - ♦ **AbstractList** → **the common behaviour**
 - ♦ **LinkedList,**
ArrayList → **specific behaviour**

Evolving Interfaces

- Add another abstract method to an interface → all classes that implement it will break.

```
public interface Matrix {  
    ...  
    void reset(); //we add a new abstract method to the interface  
}
```

- Anticipate all uses for your interface and specify it completely from the beginning!
- Create a new interface, extending the old one
 - Old classes will use the old interface or upgrade to the new interface.
- Use **default methods**.

Default Methods

- **Default methods** allow extending interfaces without breaking existing implementations.

```
public interface Matrix {  
    ...  
    default void reset() {  
        for (int i = 0; i < rowCount(); i++) {  
            for (int j = 0; j < columnCount(); j++) {  
                set(i, j, 0);  
            }  
        }  
    }  
}
```

- **Classes may override default methods**

```
public class DefaultMatrixImpl implements Matrix {  
    ...  
    @Override  
    public void reset() {  
        for (int[] row: matrix)  
            Arrays.fill(row, 0);  
    }  
}
```

Static Methods

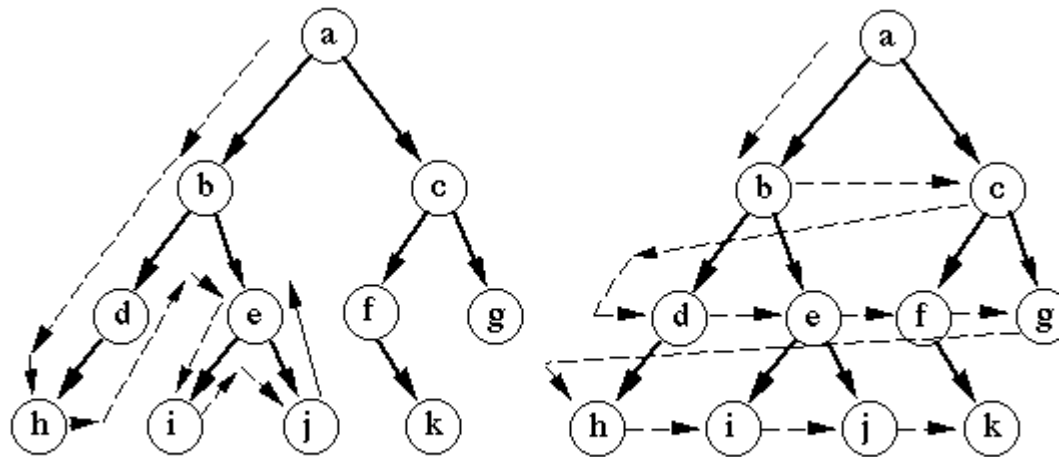
- **Static methods** allow defining specific interface methods and are identical with the ones defined in classes.
- They belong to the interface/class only.

```
interface IA {  
    static void foo() { System.out.println("IA"); }  
    static void bar() { }  
}  
interface IB extends IA {  
    static void foo() { System.out.println("IB"); }  
}  
class B implements IB {  
    static void foo() { System.out.println("B"); }  
}  
  
public static void main(String args[]) {  
    IA.foo();      IB.foo();      B.foo();      IB.bar();      B.bar();  
    IA             IB             B  
}
```

Callback Methods

A **callback** is something that you pass to a method as an argument (some piece of code).

Example: When a graph exploration algorithm reaches a node, we want to execute some kind of processing for that specific node.



Depth-first search

Breadth-first search

Implementing *Callback* Methods

```
public interface NodeProcessing {  
    public void execute(Node u);  
}
```

```
public class Graph {  
    ...  
    public void explore(NodeProcessing f) {  
        ...  
        if (exploration reached the node v) {  
            f.execute(v);  
        }  
        ...  
    }  
}
```

//Implement various processing types

```
class PrintRo implements NodeProcessing {
```

```
    public void execute(Node v) {  
        System.out.println("Nodul curent este: " + v);  
    }  
}
```

```
class PrintEn implements NodeProcessing {
```

```
    public void execute(Node v) {  
        System.out.println("The current node is: " + v);  
    }  
}
```

```
Graph g = new Graph();
```

```
...
```

```
g.explore(new PrintRo());
```

```
g.explore(new PrintEn());
```

FilenameFilter Interface

```
//Listing the files from a folder
import java.io.*;

public class ListFiles {

    public static void main ( String [] args ) {
        File folder = new File(".");
        String[] list = folder.list(new MyFilter("mp3"));
        for (int i = 0; i < list.length ; i ++ ) {
            System.out.println(list[i]);
        }
    }
}

class MyFilter implements FilenameFilter {
    String extension;

    public MyFilter (String extension) {
        this.extension = extension;
    }

    public boolean accept(File dir, String name) {
        return name.endsWith("." + extension);
    }
}
```

Anonymous Classes

Anonymous Class = Inner class used to create an object of a specific type.

```
method(new SomeInterface() {  
    // Implement interface methods  
});
```

Example:

```
folder.list(new FilenameFilter() {  
    // Anonymous class  
    public boolean accept (File dir, String name) {  
        return ( name.endsWith(".txt") );  
    }  
});
```

Compile: OuterClass\$1.class, OuterClass\$2.class, ...



Comparing Objects

```
class Person {  
    private int code;  
    private String name;  
    public Person (int code, String name ) {  
        this.code = code;  
        this.name = name ;  
    }  
    ...  
    public String toString () {  
        return code + " \t " + name;  
    }  
}
```

```
class Sorting {  
    public static void main ( String args []) {  
        Person persons[] = new Person[4];  
        persons[0] = new Person (3, " Ionescu ");  
        persons[1] = new Person (1, " Vasilescu ");  
        persons[2] = new Person (2, " Georgescu ");  
        persons[3] = new Person (4, " Popescu ");  
        java.util.Arrays.sort(persons);  
        System.out.println ("The persons were sorted...");  
        for (int i = 0; i < persons.length ; i++)  
            System.out.println (persons[i]);  
    }  
}
```



The *Comparable* Interface

Imposes a total ordering on the objects of a class (*natural*)

```
public interface Comparable {  
    int compareTo(Object o);  
}
```

```
class Person implements Comparable {  
    ...  
    public int compareTo (Object other) {  
        //returns: 0 if this==other, <0 if this<other, >0 if this>other  
  
        if (other == null ) throw new NullPointerException();  
        if (!( other instanceof Persoana ))  
            throw new ClassCastException ("Uncomparable objects!");  
        Person pers = (Person) other;  
        return (this.code - pers.code);  
    }  
}
```

```
class Person implements Comparable<Person> {  
    @Override  
    public int compareTo(Person o) {  
        ...  
    }  
}
```

The *Comparator* Interface

A comparison function, which imposes a total ordering on some collection of objects

```
import java.util.*;
class Sorting {
    public static void main ( String args []) {
        Person persons[] = new Person [4];
        persons[0] = new Person (3, " Ionescu ");
        persons[1] = new Person (1, " Vasilescu ");
        persons[2] = new Person (2, " Georgescu ");
        persons[3] = new Person (4, " Popescu ");

        Arrays.sort (persons, new Comparator<Person> () {
            public int compare (Person p1, Person p2) {
                return p1.getName().compareTo(p2.getName());
            }
        });

        System.out.println("Orderd by name:");
        System.out.println(Arrays.toString(persons));
    }
}
```

Functional Interfaces

- A functional interface is any interface that **contains only one abstract method**.
- Instead of using an anonymous class, you use a *lambda expression*, omitting the name of the interface and the name of the method.

```
Arrays.sort(persons, (Person p1, Person p2) -> {  
    return p1.getName().compareTo(p2.getName());  
});
```

```
@FunctionalInterface  
public interface Comparator { ...}
```

Method References

- Sometimes a lambda expression does nothing but call an existing method.

```
class Person {  
    ...  
    public static int compareByAge(Person a, Person b) {  
        return a.birthday.compareTo(b.birthday);  
    }  
}  
Arrays.sort(persons, (a, b) -> Person.compareByAge(a,b));
```

- In those cases, it's often clearer to refer to the existing method by name:

```
Arrays.sort(persons, Person::compareByAge);
```

Marker Interfaces

- Interfaces that do not define any method.

```
interface Serializable {}
```

```
interface Cloneable {}
```

- Their role is to associate some *metadata* to a class, that will be useful at runtime.

```
class Person
```

```
    implements Serializable, Cloneable { ... }
```

- A modern alternative: *annotations*

```
@Entity(table="persons")
```

```
class Person { ... }
```

Conclusions

- An interface defines a set of specifications
- They are essential in separating the model from the implementation.
- An interface is a common denominator between unrelated classes.
- Can be used as Reference Data Types
- Can be used to implement *Callbacks*



Advanced Programming

Generics

Collections

The Context

- “Create a data structure that stores elements:
 - *a stack, a linked list, a vector*
 - *a graph, a tree, etc.*”
- What data type to use for representing the elements of the structure?

Homogenous structure

```
class Stack {  
    private int[] items;  
    public void push (int item) { ... }  
    public int peek() { ... }  
}  
...  
Stack stack = new Stack();  
stack.push(100);  
stack.push(200);  
stack.push("Hello World!");
```

Heterogeneous structure

```
class Stack {  
    private Object[] items;  
    public void push (Object item) { ... }  
    public Object peek() { ... }  
}  
...  
Stack stack = new Stack();  
stack.push(100);  
stack.push(new Rectangle());  
stack.push("Hello World!");  
String s = (String) stack.peek;
```

Generics

- Generics enable types (classes and interfaces) to be parameters when defining classes, interfaces and methods.

```
public Stack<String> { ... }
```

- Stronger type checks at compile time.

```
stack.push(new Rectangle());
```

- Elimination of casts.

```
String s = (String) stack.peek();
```

- Enabling generic algorithms.

Defining a Generic Type

```
class ClassName<T1, T2, ..., Tn> { ... }  
or  
interface IName<T1, T2, ..., Tn> { ... }
```

```
/**  
 * A generic version of the Stack class  
 * @param <E> the type of the elements  
 */
```

```
public class Stack<E> {
```

```
    // E is a generic data type
```

```
    private E[] items;
```

```
    public void push(E item) { ... }
```

```
    public E peek() { .. }
```

```
}
```

E is the type parameter

Type Parameter Naming Conventions

- **E - Element** (used extensively by the Java Collections Framework)
- **K - Key**
- **N - Number**
- **T - Type**
- **V - Value**
- **S,U,V etc. - 2nd, 3rd, 4th types**

```
public class Node<T> { ... }
```

```
public interface Pair<K, V> { ... }
```

```
public class PairImpl<K, V> implements Pair<K, V> {...}
```

Instantiating a Generic Type

- Generic Invocation

String is the type argument

```
Stack<String> stack = new Stack<String>();
```

```
Pair<Integer,String> pair =
```

```
    new PairImpl<Integer,String>(0, "ab");
```

```
Stack<Node<Integer>> nodes = new Stack<Node<Integer>>();
```

- *The Diamond* <>

```
Stack<String> stack = new Stack<>();
```

```
Pair<Integer,String> pair = new PairImpl<>(0, "ab");
```

```
Stack<Node<Integer>> nodes = new Stack<>();
```

The compiler can determine, or infer, the type arguments from the context.

Generic Methods

Generic methods are methods that introduce their own type parameters.

```
public class Util {  
    public static <T> int countNullValues(T[] anArray) {  
        int count = 0;  
        for (T e : anArray)  
            if (e == null) {  
                ++count;  
            }  
        return count;  
    }  
}  
  
Util.countNullValues(new String[]{"a", null, "b"});  
Util.countNullValues(new Integer[]{1, 2, null, 3, null});
```

Bounded Type Parameters

```
class D <T extends A & B & C> { /* ... */ }
```

```
public class Node<T extends Number> {  
    private T t;  
    public void set(T t) { this.t = t; }  
    public T get() { return t; }
```

// Generic Method

```
public <U extends Integer> void inspect(U u) {  
    System.out.println("T: " + t.getClass().getName());  
    System.out.println("U: " + u.getClass().getName());  
}
```

```
public static void main(String[] args) {  
    Node<Double> node = new Node<>();  
    node.set(12.34); //OK  
    node.inspect(1234); //OK  
node.inspect(12.34); //compile error!  
node.inspect("some text"); //compile error!  
}
```

Generics, Inheritance, Subtypes

"is a"

- ✓ Integer extends Object
- ✓ Integer extends Number
- ~~Stack<Integer> extends Stack<Object>~~
- ~~Stack<Integer> extends Stack<Number>~~



Given two concrete types A and B (for example, *Number* and *Integer*), *MyClass<A>* has no relationship to *MyClass*, regardless of whether or not A and B are related. The common parent of *MyClass<A>* and *MyClass* is *Object*.

"This is a common misunderstanding when it comes to programming with generics, but it is an important concept to learn "

Variance

- **Variance** refers to how subtyping between more complex types relates to subtyping between their components.

```
class Animal {}  
class Cat extends Animal { }
```

```
Cat tom = new Cat();  
Animal[] pets = new Animal[10];  
Cat[] cats = new Cat[10];
```

```
tom instanceof Animal      → true  
cats instanceof Animal[] → true (covariance)  
pets instanceof Cat[]     → false (contravariance)
```

```
List<Animal> pets = new ArrayList<>();  
List<Cat> cats = new ArrayList<>();  
cats instanceof List<Animal> → compile error  
                                (illegal generic type)
```

```
void play(List<Animal> pets) { ... }  
play(cats); → compile error (incompatible types)
```

Wildcards

- Upper bounded

```
public double sumOfList(List<? extends Number> list) {  
    //it works on List<Integer>, List<Double>, List<Number>, etc.  
    double s = 0.0;  
    for (Number n : list)  
        s += n.doubleValue();  
    return s;  
}
```

- Unbounded

```
public void printList(List<?> list) {  
    for (Object elem: list)  
        System.out.print(elem + " ");  
}
```

- Lower bounded

```
public void addNumbers(List<? super Integer> list) {  
    //it works on List<Integer>, List<Number>, and List<Object> –  
    //anything that can hold Integer values.  
    for (int i = 1; i <= 10; i++) {  
        list.add(i);  
    }  
}
```



Advanced Programming

Java Collections

Framework

Collections

- A *collection* is an object that groups multiple elements into a single unit.
- *Vectors, Lists, Stacks, Sets, Dictionaries, Trees, Tables, etc.*
- Promotes software reuse
- Reduces programming effort
- Increases program speed and quality
- Benefits from *polymorphic algorithms*
- Uses *generics*

Collections Framework

- Interface



- Abstract class



- Concrete implementation

List

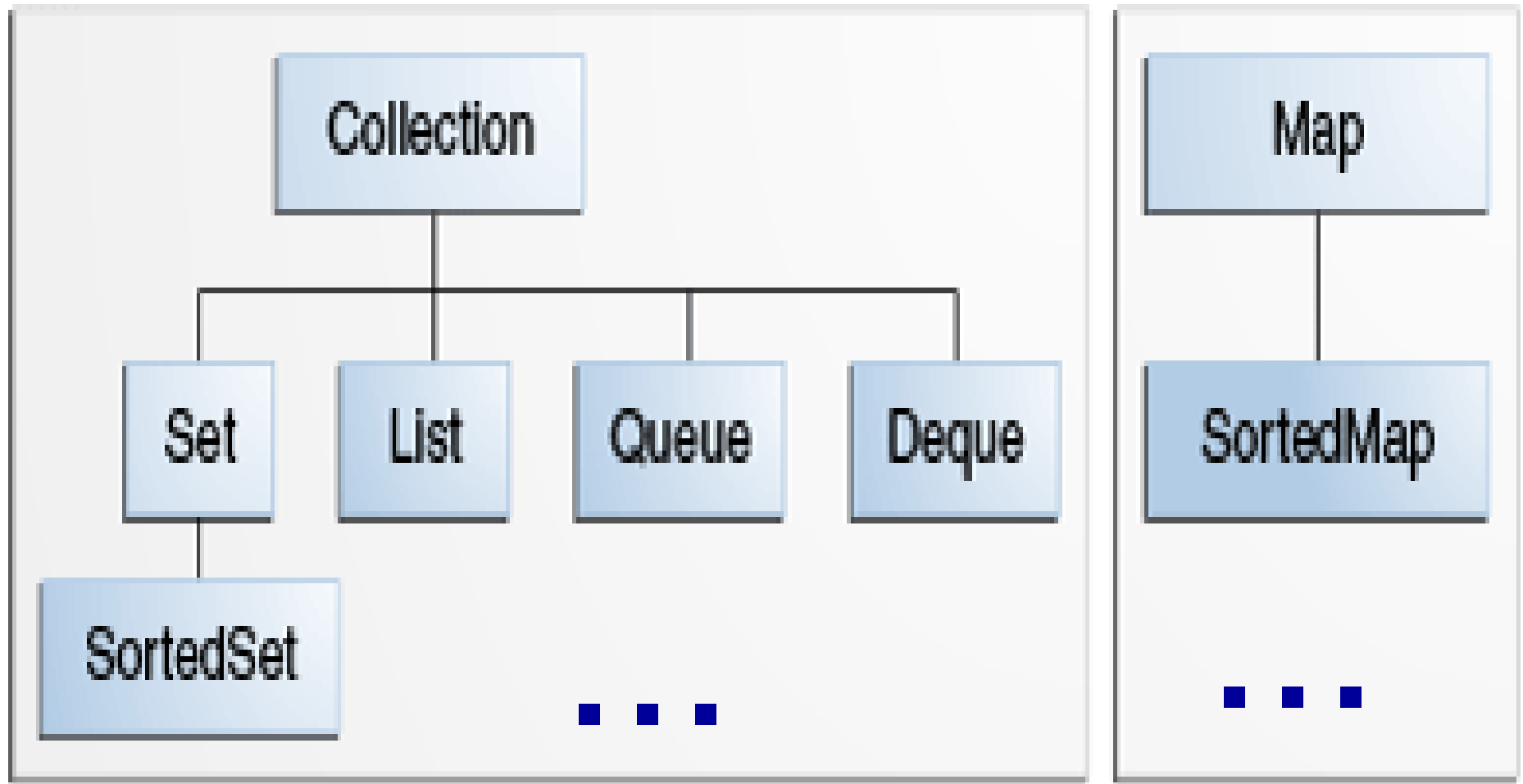


AbstractList



ArrayList
LinkedList
Vector

The Core Collection Interfaces



Implementations

Interfața	Hash	Array	Tree	Linked	Hash+Linked
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList Vector		LinkedList	
Queue		ArrayBlocking Queue		LinkedList, ConcurrentLin kedQueue	
Deque		ArrayDeque		LinkedDeque	
Map	HashMap Hashtable		TreeMap		LinkedHashMap



The *Collection* interface

- A collection represents a group of objects (elements)
- Some collections are *indexed*, others are not.
- Some collections allow *duplicate* elements, others do not.
- Some are *ordered* and others unordered.
- Some are *thread-safe*, others are not.
- All collections are *iterable*:
 - `public interface Collection<E> extends Iterable<E>`
- Dedicated subinterfaces: `List`, `Set`, `Queue`, etc.
- Many (indirect) implementing classes.

Common operations

```
public interface Collection<E> extends Iterable<E> {

    int size();
    boolean isEmpty();
    boolean contains(Object o);
    Iterator<E> iterator();

    Object[] toArray();
    <T> T[] toArray(T[] a);
    default <T> T[] toArray(IntFunction<T[]> generator) {
        return toArray(generator.apply(0));
    }

    boolean add(E e);
    boolean remove(Object o);
    boolean containsAll(Collection<?> c);
    boolean addAll(Collection<? extends E> c);
    boolean removeAll(Collection<?> c);
    boolean retainAll(Collection<?> c);
    void clear();
    ...
}
```

Lists: ArrayList, LinkedList

- A *list* is an indexed collection (sequence).
- Each element has a position (integer), starting from 0. Just like an array.
- Duplicates **are** allowed.
- A list can be implemented in various ways

```
List<String> list = new ArrayList<>();
```

```
List<Number> list = new LinkedList<>();
```

Bad practice:

Using implementations as reference types:

```
ArrayList<String> list = new ArrayList<>();
```

Using raw generics:

```
List list = new ArrayList();
```

Using a *List*

```
List<String> list = new ArrayList<>();  
  
list.add("a");  
  
list.add("b");  
  
System.out.println(list); // [a, b];
```

```
assert list.size() == 2;  
  
assert list.get(0).equals("a");  
  
assert list.get(1).equals("b");  
  
assert list.indexOf("a") == 0;
```

```
list.remove("b");
```

```
assert !list.contains("b");
```

```
assert list.contains("a");
```

indexOf, contains use the *equals* method of the elements!

Iterating Over a List

- As an indexed collections

```
for (int i=0, n=list.size(); i < n; i++ ) {  
    System.out.println( list.get(i) );  
}
```

- *Iterator and Enumeration*

```
for (Iterator it = list.iterator(); it.hasNext(); ) {  
    System.out.println(it.next());  
    it.remove();  
}
```

- *for-each*

```
List<Student> students = new ArrayList<>();  
...  
for (Student student : students) {  
    student.setGrade(10);  
}
```

Under the Hood

- **ArrayList**<E> implements **List**<E>

```
private static final int DEFAULT_CAPACITY = 10;  
Object[] elementData;  
private int size;
```

- **LinkedList**<E> implements **List**<E>

```
int size = 0;  
Node<E> first;  
Node<E> last;
```

```
private static class Node<E> {  
    E item;  
    Node<E> next;  
    Node<E> prev;  
    ...  
}
```

ArrayList or LinkedList?

```
public class TestList {
    private final static int N = 100_000;
    public void testAdd(List<Integer> list) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            list.add(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Add: " + (t2 - t1));
    }
    public void testGet(List<Integer> list) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            list.get(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Get: " + (t2 - t1));
    }
    public void testRemove(List<Integer> list) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            list.remove(0);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Remove : " + (t2 - t1));
    }
    public void runTests(List<Integer> list) {
        testAdd(list); testGet(list); testRemove(list);
    }
    public static void main(String args[]) {
        TestList app = new TestList();
        app.runTests(new ArrayList<>());
        app.runTests(new LinkedList<>());
    }
}
```

	ArrayList	LinkedList
add	6 ms	8 ms
get	3 ms	<u>4320</u> ms
remove	<u>868</u> ms	6 ms

Conclusion: Choosing a certain implementation depends on the nature of the problem being solved.

The *Set* interface

- A collection that contains no duplicate elements (models the mathematical set abstraction).

More formally, sets contain no pair of elements $e1$ and $e2$ such that $e1.equals(e2)$, and at most one null element.

- Sets are not indexed! We use *iterators* or *for-each* in order to traverse the set.
- A set can be implemented in various ways

```
Set<String> list = new HashSet<>();
```

```
Set<Number> list = new TreeSet<>();
```

Under the Hood

- **HashSet**<E> implements Set<E>

```
HashMap<E, Object> map;
```

Uses the *hashCode* of an Object.

This class offers **constant time** performance for the basic operations (add, remove, contains and size), assuming the hash function disperses the elements properly among the buckets.

- **TreeSet**<E> implements Set<E>

```
private NavigableMap<E, Object> m; //a specialized SortedMap
```

The elements are sorted using their natural ordering, or by a *Comparator* provided at set creation time

This implementation provides guaranteed **log(n) time** cost for the basic operations (add, remove and contains).

Using a Set

```
Set<String> set = new HashSet<>();  
set.add("b"); set.add("a"); set.add("a");  
System.out.println(set); // [a, b]
```

No guarantees about order.

```
assert set.size() == 2;  
assert set.get(0).equals("a");
```

```
set.remove("b");  
assert set.contains("a");
```

indexOf, contains use the *equals* method of the elements.

```
for(String s : set) {  
    System.out.println(s);  
}
```

ArrayList or HashSet?

```
public class TestSet {
    final static int N = 100_000;
    public void testAdd(Collection<Integer> collection) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            collection.add(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Add: " + (t2 - t1));
    }
    public void testIterate(Collection<?> collection) {
        long t1 = System.currentTimeMillis();
        for (Object obj : collection) {
            obj.toString(); //do something
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Iterate: " + (t2 - t1));
    }
    public void testContains(Collection<?> collection) {
        long t1 = System.currentTimeMillis();
        for (int i = 0; i < N; i++) {
            collection.contains(i);
        }
        long t2 = System.currentTimeMillis();
        System.out.println("Contains: " + (t2 - t1));
    }
    public void runTests(Collection<Integer> collection) {
        testAdd(collection); testIterate(collection); testContains(collection);
    }
    public static void main(String args[]) {
        TestSet app = new TestSet();
        app.runTests(new ArrayList<>());
        app.runTests(new HashSet<>());
    }
}
```

	ArrayList	HashSet
add	6 ms	26 ms
iterate	51 ms	12 ms
contains	<u>3997</u> ms	7 ms
<i>Memory</i>	low	high

Conclusion: Choosing a certain implementation depends on the nature of the problem being solved.

The *Queue* interface

- A queue is a first-in-first-out (**FIFO**) linear collection; It has a *head* and a *tail*.
- Provides additional *insertion*, *extraction*, and *inspection* operations.

`add, remove, element` (throw exceptions)

`offer, poll, peek` (return special values)

- Example

```
Queue<String> q = new LinkedList<>();  
q.offer("a"); // same as q.add  
q.offer("b"); // adds elements at the tail  
assert q.peek().equals("a"); //returns the head  
assert q.poll().equals("a"); //returns and removes  
assert q.peek().equals("b");
```

The *PriorityQueue* class

- A queue based on a priority **heap**. The elements of the priority queue are ordered according to their natural ordering, or by a *Comparator*. The *head* of the queue is *the least element* with respect to the ordering.
- The implementation provides **$O(\log(n))$** time for the enqueueing and dequeueing methods, **linear time** for the *remove* and *contains* methods; and **constant time** for the retrieval methods (*peek*, *element*, *size*).
- Example:

```
PriorityQueue<Integer> pQueue = new PriorityQueue<>();  
pQueue.add(3); pQueue.add(1); pQueue.add(2);  
assert pQueue.peek() .equals(1);
```

Stacks and Deques

- The *Stack* class represents a last-in-first-out (**LIFO**) stack of objects. Obsolete, just like *Vector*.
- **Deque** is a linear collection that supports element insertion and removal **at both ends** ("double ended queue").
- Deques can be used to implement LIFO collections:

```
Deque<Integer> stack = new ArrayDeque<>();  
stack.push(1); //same as addFirst  
stack.push(2);  
assert stack.peekFirst().equals(2); //same as peek  
assert stack.peekLast().equals(1);
```

The *Map* interface

- An object that maps keys to values.
- It is similar to describing a *dictionary*.
- `public interface Map<K, V> { ... }`
- A map cannot contain duplicate keys; each key can map to at most one value.
- **Example:** `Map<Country, City>`

<u>Key</u>	<u>Value</u>
Romania	→ Bucharest
France	→ Paris

- Implementations:

```
Map<Country, City> capitals = new HashMap<>();
```

```
Map<Word, List<Definition>> dict = new TreeMap<>();
```

Using a *Map*

```
Map<String, String> map = new HashMap<>();  
map.put("Romania", "Bucharest");  
map.put("France", "Paris");  
System.out.println(map);  
//Romania=Bucharest, France=Paris}  
  
assert map.get("France").equals("Paris");  
assert map.containsKey("Romania");  
  
for(String country : map.keySet()) {  
    System.out.println(map.get(country));  
}
```

Under the Hood

- *HashMap* can achieve an average time complexity of $O(1)$ for the *put* and *get* operations and space complexity of $O(n)$.
- Instead of iterating over all its elements, *HashMap* attempts to calculate the position of a value based on its key.
- *HashMap* stores elements in so-called buckets and the number of buckets is called *capacity*.

```
Node<K,V>[] table;
```

```
Set<Map.Entry<K,V>> entrySet;
```

- The *index* in the *table* array is computed using the *hashCode* method of the key object.
- Inside a bucket, values are stored either in a *list* (for less than 8 elements) or in a *balanced tree*. The data structure changes dynamically. The performance of iterating: $O(\log |\text{bucket}|)$.

hashCode and *equals*

```
public class Player {  
    private String name;  
    // ...  
  
    @Override  
    public int hashCode() {  
        int hash = 7;  
        hash = 67 * hash + Objects.hashCode(this.name);  
        return hash;  
    }  
  
    @Override  
    public boolean equals(Object obj) {  
        if (this == obj) return true;  
        if (obj == null) return false;  
        if (!(obj instanceof Player)) return false;  
        Player other = (Player) obj;  
        return Objects.equals(this.name, other.name);  
    }  
  
    // ...  
}
```

When a *hashCode()* comparison returns false, the *equals()* method must also return false.

The importance of *equals*

Assume there is another property *Integer id*;

```
public boolean equals(Object obj) {  
    if (this == obj) return true;  
    if (obj == null) return false;  
    if (!(obj instanceof Player)) return false;  
    Player pers = (Player) other;  
    return Objects.equals(this.id, other.id);  
}
```

```
Map<Player, String> map = new HashMap<>();  
map.put(new Player("Messi"), "Barcelona");  
map.put(new Player("Ronaldo"), "Juventus");
```

```
System.out.println(map);
```

```
{Messi=Juventus}
```

Polymorphic Algorithms

`java.util.Collections`

- `sort`
- `shuffle`
- `binarySearch`
- `reverse`
- `fill`
- `copy`
- `min`
- `max`
- `swap`
- `enumeration`
- `unmodifiableCollectionType`

```
List<String> immutablelist = Collections.unmodifiableList(list);  
immutablelist.add("Oops...?!");
```
- `synchronizedCollectionType`



What DesignPattern?

var

Type inference for local variables

- The explicit type can be replaced by the reserved type name **var** for local variable declarations that have initializers.

```
String str = "Hello";
```



```
var str = "Hello World";
```

- Explicit vs. Implicit, Verbosity vs. Readability
- Examples

```
var stringList = List.of("a", "b", "c");
```

```
var personList = new ArrayList<Person>();  
for(var person : personList) { }
```

```
var companyToEmployees = new HashMap<String, List<String>>();  
for (var entry: companyToEmployees.entrySet()) {  
    var employees = entry.getValue();  
}
```

```
var itemQueue = new PriorityQueue<>(); // PriorityQueue<Object>  
var x = null;
```